

GUIA DE ESTUDO

# Entrega Contínua e Implantação

*Blue-Green, Canary, Rolling Update e Feature Flags*

Um guia completo e passo a passo, do zero, sobre estratégias modernas de implantação de software.

Material de apoio educacional — Entrega Contínua & Implantação

# Sumário

# 1. Introdução: por que este assunto importa

---

Se você está vendo este tema pela primeira vez, a primeira pergunta é: por que existe tanta engenharia em torno de "como colocar código em produção"? A resposta curta é: porque o momento do deploy é, historicamente, o momento de maior risco no ciclo de vida de um software. É quando a teoria (código que passou nos testes) encontra a realidade (usuários reais, tráfego real, dados reais). Este guia explica, do zero, os conceitos e as técnicas que a indústria desenvolveu para tornar esse momento seguro, prático e reversível.

## 1.1 O pipeline moderno: do commit ao usuário

Quando um desenvolvedor escreve código, esse código passa por uma sequência de etapas até chegar ao usuário final. Entender essa sequência é o primeiro passo para entender por que as estratégias de implantação existem.

1. **Desenvolvimento:** o código é escrito localmente pelo desenvolvedor.
2. **Versionamento:** o código é enviado (commit/push) para um repositório compartilhado, como o Git.
3. **Integração contínua (CI):** um sistema automatizado baixa o código, compila, roda testes automatizados e gera um artefato (ex.: uma imagem Docker).
4. **Entrega contínua (CD):** esse artefato é considerado "pronto para produção", mas ainda não foi enviado a ela.
5. **Implantação (Deployment):** o artefato é efetivamente colocado no ambiente de produção, onde usuários reais irão utilizá-lo.

É exatamente nessa última etapa — a implantação — que entram as quatro estratégias estudadas neste guia: Rolling Update, Blue-Green, Canary e Feature Flags. Cada uma resolve, de um jeito diferente, o mesmo problema fundamental: como trocar a versão de um software em produção sem quebrar a experiência do usuário, e com a possibilidade de reverter rapidamente se algo der errado.

## 1.2 CI, CD e Continuous Deployment: três siglas que se confundem

É muito comum confundir esses três termos, especialmente porque todos usam as siglas CI/CD. Vamos destrinchar cada um com cuidado, pois eles representam níveis crescentes de automação.

### Continuous Integration (CI) — Integração Contínua

É a prática de integrar o código de diferentes desenvolvedores com muita frequência (idealmente, várias vezes ao dia) em um repositório compartilhado. A cada integração, um pipeline automatizado compila o projeto e executa os testes automatizados (unitários, de integração etc.). O objetivo é detectar erros de integração o mais cedo possível — quando ainda são baratos e fáceis de corrigir.

**Analogia:** imagine que vários autores estão escrevendo capítulos diferentes de um mesmo livro. Se cada um só mostrar seu capítulo no final, é provável que os estilos e a continuidade da história não combinem. Se, ao invés disso, cada capítulo for revisado e encaixado no livro assim que fica pronto, problemas de coerência aparecem cedo e são fáceis de resolver.

### Continuous Delivery (CD) — Entrega Contínua

É a extensão natural da Integração Contínua: além de integrar e testar o código automaticamente, o pipeline também prepara um artefato "pronto para ir para produção" a qualquer momento. A decisão de efetivamente colocar essa versão em produção, porém, ainda é manual — normalmente um clique de aprovação de alguém do time (um gerente de release, um líder técnico etc.).

### Continuous Deployment (CD+) — Implantação Contínua

É o nível mais avançado de automação: toda mudança que passa em todos os testes automatizados vai para produção automaticamente, sem intervenção humana. Não existe um botão de "aprovar" — a confiança está totalmente depositada na suíte de testes e nos mecanismos de segurança do pipeline.

#### ⚠ Atenção à confusão comum

"Continuous Delivery" e "Continuous Deployment" são frequentemente abreviados da mesma forma (CD), mas são conceitos diferentes. Delivery = pronto para produção, aguardando aprovação humana. Deployment = vai para produção automaticamente. Fique atento ao contexto ao ler materiais em inglês.

| Nível                  | O que é automatizado                    | Decisão de ir para produção        |
|------------------------|-----------------------------------------|------------------------------------|
| Continuous Integration | Build + testes automatizados            | Não se aplica (ainda não é deploy) |
| Continuous Delivery    | Build + testes + empacotamento          | Manual (aprovação humana)          |
| Continuous Deployment  | Build + testes + empacotamento + deploy | Automática                         |

## 1.3 O problema do "big bang deploy"

Antes dessas técnicas se popularizarem, era comum que equipes fizessem o que se chama informalmente de "big bang deploy": desligar a versão antiga por completo, instalar a nova versão em todos os servidores de uma vez, e ligar tudo novamente. Esse modelo tem três problemas sérios:

- **Indisponibilidade (downtime):** entre desligar a versão antiga e ligar a nova, o sistema fica fora do ar — inaceitável para a maioria dos serviços modernos, que precisam estar disponíveis 24 horas por dia.
- **Risco concentrado:** se a nova versão tiver um bug crítico, 100% dos usuários são afetados imediatamente e ao mesmo tempo.
- **Rollback lento e estressante:** reverter para a versão anterior exige repetir todo o processo de desligar/instalar/ligar, sob pressão, enquanto o sistema já está com problemas em produção.

As quatro estratégias estudadas neste guia nasceram exatamente para resolver esses três problemas, cada uma com uma abordagem diferente e um conjunto próprio de vantagens e desvantagens.



## 2. Conceitos fundamentais antes de começar

---

Antes de estudar cada estratégia em detalhe, é importante dominar um pequeno vocabulário técnico que aparece repetidamente. Se este é seu primeiro contato com o tema, recomendamos ler esta seção com atenção antes de avançar.

### 2.1 Deploy vs. Release — uma distinção crucial

Esses dois termos parecem sinônimos, mas a diferença entre eles é a base para entender por que Feature Flags existem.

- **Deploy:** é o ato técnico de colocar uma nova versão do código em execução em um ambiente (por exemplo, em produção). O código está rodando nos servidores.
- **Release:** é o ato de tornar uma funcionalidade visível/disponível para os usuários finais.

Tradicionalmente, deploy e release aconteciam ao mesmo tempo: assim que o código ia para produção, os usuários já viam a mudança. Com Feature Flags, esses dois eventos podem ser separados: o código pode estar em produção (deploy feito), mas a funcionalidade continuar invisível para os usuários (release ainda não feito) até que alguém decida "ligar" a flag.

### 2.2 Downtime, Zero-downtime e SLA

- **Downtime:** período em que o sistema fica indisponível ou não responde corretamente às requisições dos usuários.
- **Zero-downtime deployment:** uma implantação realizada de forma que os usuários não percebam nenhuma interrupção do serviço.
- **SLA (Service Level Agreement):** um acordo formal (contratual ou interno) que define o nível mínimo de disponibilidade esperado de um sistema, geralmente expresso em porcentagem (ex.: 99,9% de disponibilidade ao ano, o que permite pouco menos de 9 horas de indisponibilidade por ano).

### 2.3 Rollback e Rollforward

- **Rollback:** reverter o sistema para uma versão anterior, considerada estável, geralmente porque a versão nova apresentou um problema.
- **Rollforward (ou "fix forward"):** em vez de voltar para trás, corrige-se o problema e avança-se com uma nova versão corrigida. É uma alternativa ao rollback, mais comum quando o rollback é tecnicamente difícil (por exemplo, quando já houve alterações no banco de dados que não podem ser desfeitas facilmente).

A velocidade e a facilidade do rollback é um dos principais critérios usados para comparar as quatro estratégias deste guia — por isso ele aparece com destaque nas tabelas comparativas mais adiante.

### 2.4 Observabilidade: métricas, logs e alertas

Observabilidade é a capacidade de entender o que está acontecendo dentro de um sistema a partir de dados que ele expõe externamente. Ela é especialmente importante para a estratégia Canary, que depende de decisões baseadas em dados em tempo real. Os três pilares clássicos da observabilidade são:

- **Métricas:** números agregados ao longo do tempo, como taxa de erros (% de requisições que falham), latência (tempo de resposta) e throughput (requisições por segundo).
- **Logs:** registros textuais e detalhados de eventos individuais que aconteceram no sistema, úteis para investigar um problema específico.
- **Alertas:** regras automatizadas que notificam a equipe quando uma métrica ultrapassa um limite aceitável (ex.: "taxa de erro acima de 5%").

## 2.5 Health checks (verificações de saúde)

Um health check é um mecanismo pelo qual o orquestrador de infraestrutura (como o Kubernetes) pergunta periodicamente a uma instância da aplicação: "você está funcionando corretamente?". Existem dois tipos principais, muito usados no contexto de Rolling Update:

- **Liveness probe:** verifica se a aplicação ainda está viva (não travou). Se falhar, a instância é reiniciada.
- **Readiness probe:** verifica se a aplicação está pronta para receber tráfego (por exemplo, já terminou de carregar suas configurações e se conectar ao banco de dados). Se falhar, a instância é temporariamente removida do balanceamento de carga, mas não é reiniciada.

Esses health checks são o que permite ao Rolling Update saber quando é seguro prosseguir para a próxima instância — sem eles, a estratégia não teria como detectar que uma nova versão está com problemas.

## 3. Rolling Update (Atualização Gradual)

### 3.1 O que é

Rolling Update é a estratégia de implantação mais difundida e é o comportamento padrão de orquestradores de contêineres como o Kubernetes. A ideia central é simples: em vez de trocar todas as instâncias da aplicação de uma vez, o sistema troca poucas instâncias por vez, de forma gradual e controlada, até que toda a frota esteja rodando a nova versão.

### 3.2 Como funciona, passo a passo

Imagine uma aplicação rodando em 6 instâncias (também chamadas de réplicas ou pods, no jargão do Kubernetes), todas na versão v1. Queremos atualizar para a versão v2. O processo típico acontece assim:

1. O orquestrador seleciona um pequeno lote de instâncias antigas (por exemplo, 1 ou 2) para atualizar.
2. Uma nova instância com a versão v2 é criada.
3. O orquestrador aguarda o readiness probe da nova instância confirmar que ela está pronta para receber tráfego.
4. Somente depois da confirmação, uma instância antiga (v1) é removida do balanceamento de carga e desligada.
5. O processo se repete para o próximo lote, até que todas as 6 instâncias estejam na versão v2.

Durante todo esse processo — que pode durar de segundos a vários minutos, dependendo do tamanho da frota e da velocidade configurada — as versões v1 e v2 coexistem em produção, atendendo tráfego real simultaneamente. Isso é importante: se v1 e v2 tiverem comportamentos incompatíveis (por exemplo, formatos de dados diferentes), pode haver inconsistências durante a transição.

#### Parâmetros importantes (exemplo do Kubernetes)

- **maxUnavailable:** quantas instâncias podem ficar indisponíveis ao mesmo tempo durante a atualização. Um valor maior acelera o processo, mas reduz a capacidade disponível durante a transição.
- **maxSurge:** quantas instâncias extras (acima da quantidade normal) podem ser criadas temporariamente para acelerar a transição sem reduzir a capacidade.

### 3.3 Vantagens e desvantagens

| ✓ Vantagens                                                                                                                                                                                                                                           | ✗ Desvantagens                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Simples de configurar e entender — é o padrão de praticamente todo orquestrador moderno.</li><li>• Não exige infraestrutura duplicada, então o custo de infraestrutura permanece o mesmo de sempre.</li></ul> | <ul style="list-style-type: none"><li>• O rollback também é gradual — reverter para a versão anterior leva tempo, pois segue o mesmo processo passo a passo, só que ao contrário.</li><li>• Durante a transição, duas versões diferentes do software convivem em produção ao mesmo tempo, o que exige que elas sejam compatíveis entre si.</li></ul> |

- É totalmente automatizável e nativo em ferramentas como Kubernetes, sem necessidade de ferramentas adicionais.

- Não oferece controle fino sobre o tráfego — não é possível, por exemplo, direcionar apenas 5% dos usuários para a nova versão; o controle é apenas sobre quantas instâncias são atualizadas.

### 3.4 Quando usar

✓ **Use Rolling Update quando:**

a equipe busca simplicidade operacional, o orçamento não permite infraestrutura duplicada, as versões antiga e nova são compatíveis entre si durante a transição, e não há necessidade de controle fino de porcentagem de tráfego.

## 4. Blue-Green Deployment

### 4.1 Origem do nome e conceito

O nome vem de uma analogia simples: imagine dois ambientes de produção completos e idênticos, identificados por cores — "Blue" e "Green". A qualquer momento, apenas um deles está efetivamente recebendo o tráfego dos usuários (digamos, o Blue). O outro (Green) fica disponível para receber a nova versão do software e ser testado exaustivamente, sem nenhum risco, já que nenhum usuário real está passando por ele.

### 4.2 Como funciona, passo a passo

1. O ambiente Blue está ativo, atendendo 100% do tráfego de produção com a versão v1.
2. A nova versão (v2) é implantada por completo no ambiente Green, que está tecnicamente pronto, mas ainda não recebe tráfego de usuários reais.
3. A equipe testa o ambiente Green de forma isolada e completa: testes automatizados, testes manuais, testes de carga, validações de negócio etc.
4. Quando o Green é considerado validado, um componente de roteamento (um load balancer, um proxy reverso ou até uma alteração de DNS) é reconfigurado para direcionar 100% do tráfego, de uma só vez, do Blue para o Green.
5. O Blue permanece de pé, ocioso, servindo como plano de contingência. Se algo der errado com o Green, o tráfego pode ser instantaneamente redirecionado de volta para o Blue.
6. Depois de um período de observação sem problemas, o ambiente Blue pode ser desligado ou reaproveitado para a próxima rodada (nesse ponto, os papéis se invertem: Green se torna o novo "ambiente estável" e o antigo Blue se torna o próximo ambiente de teste).

### O desafio das migrações de banco de dados

O ponto mais delicado do Blue-Green é quando a nova versão exige mudanças na estrutura do banco de dados (schema). Como os dois ambientes normalmente compartilham o mesmo banco (duplicar bancos de dados é caro e complexo de sincronizar), é preciso garantir que o schema do banco seja compatível simultaneamente com a versão antiga e com a nova, pelo menos durante a janela de transição. Uma técnica comum é dividir mudanças de schema em etapas "aditivas" (que não quebram a versão antiga) antes de fazer o switch, e só remover estruturas antigas depois que o Blue for definitivamente desativado.

### 4.3 Vantagens e desvantagens

| ✓ Vantagens                                                                                                                                                          | ✗ Desvantagens                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Rollback instantâneo: basta apontar o roteamento de volta para o ambiente anterior, sem precisar reverter deploys.</li></ul> | <ul style="list-style-type: none"><li>• Custo de infraestrutura dobrado, já que é preciso manter dois ambientes de produção completos, mesmo que um fique ocioso na maior parte do tempo.</li></ul> |

- A nova versão é validada de forma completa e isolada antes de qualquer usuário real ser exposto a ela.
- Não há período de indisponibilidade no momento da troca (a troca de roteamento é praticamente instantânea).
- Sincronizar dados e schema de banco de dados entre os dois ambientes é uma fonte comum de complexidade e erros.
- A troca é do tipo "tudo ou nada": não existe uma forma nativa de expor a nova versão gradualmente para uma fração dos usuários.

## 4.4 Quando usar

### ✓ Use Blue-Green quando:

a capacidade de reverter instantaneamente é crítica para o negócio, a equipe pode arcar com o custo de manter infraestrutura duplicada (mesmo que temporariamente), e é importante testar a nova versão por completo, isolada de usuários reais, antes de liberá-la.

## 5. Canary Release

### 5.1 Origem do nome

O nome é uma referência a uma prática histórica de mineradores de carvão, que levavam canários (pássaros) para dentro das minas. Canários são particularmente sensíveis a gases tóxicos como o monóxido de carbono; se o pássaro desmaiasse ou morresse, isso servia como um alerta precoce para que os mineiros evacuassem antes que o gás os afetasse a eles. Da mesma forma, na estratégia Canary de software, um pequeno grupo de usuários (o "canário") é exposto à nova versão primeiro, servindo como um sistema de alerta precoce para o resto da base de usuários.

### 5.2 Como funciona, passo a passo

1. A versão v1 (estável) está em produção, recebendo 100% do tráfego.
2. A versão v2 é implantada em paralelo, mas o roteador é configurado para enviar apenas uma pequena fatia do tráfego real para ela — por exemplo, 5%.
3. Enquanto isso, o time monitora de perto métricas-chave: taxa de erros, latência, uso de recursos e, frequentemente, métricas de negócio (como taxa de conversão ou de checkout).
4. Se as métricas da v2 permanecem saudáveis (dentro dos limites aceitáveis) por um período de observação, o percentual de tráfego direcionado a ela aumenta gradualmente — por exemplo, seguindo uma progressão como 5% → 25% → 50% → 100%.
5. Se a qualquer momento as métricas da v2 piorarem significativamente, o tráfego é revertido para 0% (voltando tudo para v1), e a implantação é interrompida para investigação.
6. Quando a v2 atinge 100% do tráfego e se mostra estável, o processo é considerado concluído, e a v1 pode ser desativada.

#### Canary manual vs. automatizado

Em times menos maduros, os incrementos de tráfego e a decisão de avançar ou reverter são feitos manualmente por um engenheiro observando dashboards. Em times mais avançados, ferramentas especializadas (como o Argo Rollouts, mencionado no Capítulo 8) automatizam esse processo por completo: elas consultam automaticamente o sistema de métricas, aplicam regras de aprovação/reprovação (ex.: "se a taxa de erro subir mais de 1 ponto percentual em relação à v1, reverter automaticamente") e avançam os incrementos de tráfego sem intervenção humana.

### 5.3 Vantagens e desvantagens

| ✓ Vantagens                                                                                                                                                                            | ✗ Desvantagens                                                                                                                                                                                                                                                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Risco bastante reduzido: se a nova versão tiver um problema, ele afeta apenas uma pequena fração dos usuários, não todos de uma vez.</li></ul> | <ul style="list-style-type: none"><li>• Exige um nível de observabilidade robusto — sem boas métricas e alertas, é impossível saber se o canário está "passando mal".</li><li>• O roteamento de tráfego por porcentagem é tecnicamente mais sofisticado do que</li></ul> |

- As decisões de avançar ou reverter são baseadas em dados reais de produção, não apenas em testes pré-produção.
- Permite validar a nova versão com o comportamento real dos usuários, algo que ambientes de teste nem sempre conseguem simular fielmente.

simplesmente ligar/desligar um ambiente inteiro.

- O processo até chegar a 100% do tráfego é mais lento do que outras estratégias, já que depende de períodos de observação em cada incremento.

## 5.4 Quando usar

### ✓ Use Canary quando:

o time já possui (ou está disposto a investir em) uma boa infraestrutura de observabilidade, o produto tem volume de tráfego suficiente para que uma fração pequena ainda seja estatisticamente significativa, e a equipe quer validar mudanças de alto risco com usuários reais antes de um rollout completo.

## 6. Feature Flags (Feature Toggles)

---

### 6.1 O conceito central: desacoplar deploy de release

Diferente das três estratégias anteriores, que atuam no nível de infraestrutura (quais instâncias/ambientes recebem tráfego), Feature Flags atuam no nível do próprio código da aplicação. A ideia é envolver um trecho de código novo em uma condição ("se a flag X estiver ativada, execute o código novo; senão, execute o código antigo"), e essa condição pode ser ligada ou desligada em tempo real, sem necessidade de um novo deploy.

Isso permite, como vimos na Seção 2.1, separar completamente o momento do deploy (código chega em produção) do momento do release (funcionalidade se torna visível para os usuários). O código de uma funcionalidade nova pode estar em produção há semanas, "desligado", até que a equipe decida o momento certo de ativá-la.

### 6.2 Tipos de Feature Flags

Nem toda flag serve ao mesmo propósito. É comum classificá-las em quatro categorias:

#### Release toggles

Usadas para esconder funcionalidades que ainda estão em desenvolvimento, permitindo que o código seja integrado (merge) ao branch principal continuamente, mesmo antes de a funcionalidade estar pronta para os usuários. Exemplo: uma nova tela de checkout está sendo construída ao longo de várias semanas; a flag mantém essa tela invisível até que esteja completa.

#### Experiment toggles

Usadas para testes A/B: diferentes grupos de usuários veem variações diferentes de uma funcionalidade, e métricas de comportamento são comparadas entre os grupos para decidir qual variação é melhor. Exemplo: metade dos usuários vê um botão azul, a outra metade vê um botão verde, e a equipe mede qual gera mais cliques.

#### Ops toggles (interruptores operacionais)

Funcionam como um "disjuntor de emergência": permitem desligar rapidamente uma funcionalidade que está causando problemas operacionais (por exemplo, sobrecarregando um banco de dados), sem precisar reverter um deploy inteiro. Exemplo: desligar temporariamente um recurso de exportação de relatórios muito pesado durante um pico de tráfego.

#### Permission toggles (flags de permissão)

Usadas para liberar funcionalidades apenas para determinados usuários, planos de assinatura ou grupos. Exemplo: um recurso avançado de analytics fica disponível apenas para clientes do plano "Enterprise".

### 6.3 O ciclo de vida de uma flag

Uma boa prática frequentemente negligenciada é que toda flag deve ter um ciclo de vida definido, com um fim planejado. O fluxo saudável costuma ser:

1. Criação da flag e do código condicional associado a ela.
2. Ativação gradual (ex.: internamente na empresa, depois para um pequeno grupo de usuários, depois para todos).
3. Quando a funcionalidade está estável e 100% ativa para todos, a flag deveria ser removida do código, junto com o caminho antigo que ela substituiu.

#### ⚠ O risco da dívida técnica

O passo 3 do ciclo de vida acima é o mais frequentemente ignorado na prática. Quando isso acontece repetidamente, o código acumula dezenas ou centenas de flags obsoletas ("flags esquecidas"), tornando-o mais difícil de entender, testar e manter — cada flag esquecida representa, na prática, um caminho de código a mais para testar e raciocinar.

## 6.4 Vantagens e desvantagens

| ✓ Vantagens                                                                                                                                                                                                                                                                                                                                                                                                                                           | ✗ Desvantagens                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• Rollback instantâneo de uma funcionalidade específica, sem precisar reverter um deploy inteiro — basta desligar a flag.</li> <li>• Permite testes A/B e liberações segmentadas por usuário, grupo ou plano, com controle muito granular.</li> <li>• Deploy e release se tornam eventos completamente independentes, dando flexibilidade ao negócio para escolher o melhor momento de lançar algo.</li> </ul> | <ul style="list-style-type: none"> <li>• Gera dívida técnica quando as flags não são removidas após cumprirem seu propósito.</li> <li>• Aumenta a complexidade dos testes, já que existem múltiplos caminhos possíveis de código dependendo do estado das flags.</li> <li>• Exige um serviço ou mecanismo dedicado para gerenciar o estado das flags de forma confiável e com baixa latência.</li> </ul> |

## 6.5 Quando usar

#### ✓ Use Feature Flags quando:

a equipe precisa de controle granular sobre quem vê o quê (por usuário, grupo ou plano), quer separar o momento técnico do deploy do momento de negócio do lançamento, ou precisa de um mecanismo de "desligamento de emergência" rápido para funcionalidades específicas.

## 7. Comparativo consolidado das quatro estratégias

Depois de estudar cada estratégia em profundidade, é útil compará-las lado a lado. Não existe uma estratégia "melhor" de forma absoluta — a escolha depende do contexto: risco aceitável, orçamento de infraestrutura, maturidade de observabilidade da equipe e requisitos do produto.

| Estratégia     | Velocidade de rollback | Custo de infraestrutura | Controle de tráfego       | Complexidade |
|----------------|------------------------|-------------------------|---------------------------|--------------|
| Rolling Update | Médio (gradual)        | Baixo                   | Nenhum                    | Baixa        |
| Blue-Green     | Instantâneo            | Alto (dobrado)          | Total (tudo ou nada)      | Média        |
| Canary         | Rápido                 | Médio                   | Fino (percentual gradual) | Alta         |
| Feature Flags  | Instantâneo            | Baixo                   | Por usuário/grupo         | Média        |

### 7.1 Um guia rápido de decisão

- **Se o orçamento é limitado e a equipe quer simplicidade:** comece com Rolling Update.
- **Se rollback instantâneo é crítico e há orçamento para infraestrutura duplicada:** considere Blue-Green.
- **Se o produto tem tráfego significativo e a equipe já tem boa observabilidade:** considere Canary para reduzir o risco de mudanças grandes.
- **Se a necessidade é de controle granular por usuário/grupo, ou separar deploy de release por razões de negócio:** use Feature Flags.



#### Na prática, essas estratégias se combinam

É muito comum, em ambientes maduros, ver combinações: por exemplo, um Canary automatizado usado dentro de uma infraestrutura Blue-Green, com Feature Flags controlando funcionalidades específicas dentro de ambas. Elas não são mutuamente exclusivas — são ferramentas complementares na caixa de ferramentas de um time de engenharia.

## 8. Panorama de tecnologias e ferramentas

---

Esta seção apresenta as principais ferramentas de mercado usadas para implementar cada estratégia. Elas se dividem em três grandes categorias: orquestração/plataformas de deploy, roteamento de tráfego (service mesh) e plataformas de feature flags.

### 8.1 Orquestração e plataformas de deploy

#### Kubernetes nativo

O Kubernetes é a plataforma de orquestração de contêineres mais usada atualmente. O objeto padrão para gerenciar aplicações, chamado Deployment, já implementa Rolling Update nativamente, sem necessidade de nenhuma ferramenta adicional. É a opção mais simples e acessível para começar.

#### Argo Rollouts

É uma extensão (controlador customizado) para Kubernetes especializada em estratégias avançadas de deploy: implementa Blue-Green e Canary de forma nativa, incluindo integração com sistemas de métricas para automatizar decisões de avançar ou reverter. Exige a instalação de CRDs (Custom Resource Definitions) e alguma configuração adicional em relação ao Kubernetes padrão.

#### Spinnaker

Uma plataforma de entrega contínua multi-cloud, originalmente desenvolvida pela Netflix, voltada para pipelines de deploy complexos e sofisticados. É bastante usada em grandes empresas, mas tem uma configuração inicial (setup) considerada pesada para times pequenos.

#### AWS CodeDeploy e Azure DevOps

Serviços de deploy gerenciados pelos respectivos provedores de nuvem (Amazon Web Services e Microsoft Azure), que oferecem Blue-Green e Canary como recursos prontos, com menor esforço operacional — mas amarram a solução ao ecossistema daquele provedor específico de nuvem.

### 8.2 Roteamento de tráfego (Service Mesh)

Essas ferramentas são especialmente relevantes para implementar Canary, já que ele exige a capacidade de dividir o tráfego entre versões por porcentagem.

#### Istio

Um service mesh completo e muito usado, com recursos avançados de roteamento — inclusive por porcentagem de tráfego e por cabeçalhos HTTP específicos (por exemplo, rotear apenas usuários com um determinado cookie para a nova versão).

#### Linkerd

Também é um service mesh, mas com foco em ser mais leve e simples operacionalmente do que o Istio, sacrificando um pouco da flexibilidade em troca de menor complexidade.

#### Nginx e Traefik

Não são service meshes completos, mas servidores/proxies reversos amplamente usados que também suportam roteamento por peso (porcentagem) através de configuração relativamente simples. São frequentemente a porta de entrada mais acessível para quem está aprendendo ou montando uma prova de conceito, por não exigirem a complexidade de um service mesh completo.

## 8.3 Plataformas de Feature Flags

### Unleash

Uma plataforma de feature flags de código aberto (open-source), que pode ser hospedada pela própria equipe (self-hosted), por exemplo via Docker. É gratuita e sobe rapidamente, o que a torna uma ótima opção para aprendizado e provas de conceito.

### Flagsmith

Também open-source e self-hosted, com painel web e SDKs (bibliotecas de integração) para múltiplas linguagens de programação.

### LaunchDarkly

Uma referência de mercado no espaço de feature flags, com recursos avançados voltados a grandes empresas (enterprise), mas é um serviço pago (SaaS — Software as a Service), sem opção de hospedagem própria.

### Split.io

Outra plataforma paga, com forte foco em experimentação e testes A/B, além do controle básico de feature flags.

## 8.4 Tabela comparativa de plataformas de Feature Flags

| Critério                   | Unleash          | Flagsmith     | LaunchDarkly        | Split.io       |
|----------------------------|------------------|---------------|---------------------|----------------|
| Licença                    | Open-source      | Open-source   | Comercial           | Comercial      |
| Self-hosted                | Sim (Docker)     | Sim (Docker)  | Não (SaaS)          | Não (SaaS)     |
| Custo para aprendizado/PoC | Gratuito         | Gratuito      | Trial limitado      | Trial limitado |
| Facilidade de setup local  | Muito rápida     | Rápida        | Não aplicável       | Não aplicável  |
| Foco principal             | Toggles + regras | Toggles + A/B | Recursos enterprise | Experimentação |

Para fins de aprendizado e provas de conceito, ferramentas open-source e self-hosted (como Unleash, Flagsmith, Nginx/Traefik e o próprio Kubernetes) tendem a ser as mais acessíveis, por não exigirem contas pagas, aprovações de orçamento ou dependência de um provedor de nuvem específico.

## 9. Casos reais de empresas

---

Tudo o que foi apresentado até aqui não é apenas teoria acadêmica — é a forma como grandes empresas de tecnologia realmente operam seus sistemas em produção. Esta seção traz exemplos documentados publicamente por essas empresas (em blogs técnicos oficiais e registros regulatórios), mostrando cada estratégia em ação no mundo real, além de um caso de alerta sobre o que acontece quando essas práticas não existem.

### 9.1 Netflix e o Canary Release automatizado

A Netflix é provavelmente o exemplo mais citado de uso avançado de Canary Release. A empresa desenvolveu o Kayenta, uma plataforma de Análise Automatizada de Canário (Automated Canary Analysis), disponibilizada como código aberto em 2018 em parceria com o Google. O Kayenta funciona integrado ao Spinnaker (a plataforma de entrega contínua também criada pela Netflix, mencionada na Seção 8.1): ele compara automaticamente métricas — como códigos de status HTTP, tempo de resposta, contagem de exceções e uso de CPU — entre um cluster "baseline" (a versão estável) e um cluster "canário" (a nova versão), decidindo sozinho se o rollout deve continuar, ser revertido, ou se precisa de intervenção manual.

Antes dessa automação, a análise de canário na Netflix era feita manualmente: um engenheiro observava gráficos e logs dos servidores baseline e canário para julgar se os dados eram parecidos o suficiente para prosseguir. A própria empresa reconhece que seu processo de deploy já foi bem mais doloroso do que é hoje — em um relato sobre a evolução de suas práticas de operação, a equipe de engenharia descreveu que, em 2012, os deploys eram tão difíceis que pareciam "caminhar na areia molhada".

### 9.2 Amazon (AWS) e o Blue-Green Deployment em escala

A Amazon, através da AWS, oferece Blue-Green Deployment como um recurso de primeira classe em vários de seus serviços gerenciados — não apenas como um conceito teórico, mas como um fluxo de trabalho automatizado. No Amazon ECS (Elastic Container Service), por exemplo, o processo de Blue-Green segue seis fases bem definidas: preparação do ambiente green, deploy da nova versão nesse ambiente, validação com tráfego de teste, transição gradual do tráfego de produção, monitoramento durante um período de observação ("bake time") e, por fim, finalização (desligando o ambiente blue ou mantendo-o disponível para rollback).

Um detalhe interessante da implementação da AWS é que ela combina conceitos: usando o serviço de DNS Route 53 com roteamento por peso, a transição de tráfego entre o ambiente blue e o green pode ser feita de forma gradual (por exemplo, aumentando de 10% em 10% o tráfego direcionado ao green) em vez de uma troca instantânea — misturando, na prática, elementos de Blue-Green com elementos de Canary.

### 9.3 Etsy e Facebook: a origem prática dos Feature Flags

O conceito de Feature Flags foi popularizado, no fim dos anos 2000, por um pequeno grupo de empresas que hoje são consideradas pioneiras da cultura de entrega contínua. O Flickr começou a falar publicamente sobre o padrão de "feature flippers" em 2009, e a Etsy documentou publicamente, a partir de 2011, como o que chamavam de "config flags" as ajudava a escalar sua engenharia. O então CTO da Etsy, Chad Dickerson,

descreveu a prática afirmando que boa parte do código de novas funcionalidades passava dias ou semanas rodando "no escuro" em produção antes de ser exposto aos usuários.

O Facebook, por sua vez, é creditado por ter popularizado o termo "dark launch" (lançamento às escuras) em 2008, durante o desenvolvimento do Facebook Chat. Semanas antes do lançamento oficial, a equipe de engenharia já fazia as páginas do Facebook se conectarem aos servidores de chat (escritos em Erlang) e simularem o envio de mensagens e consultas de status "online/offline" em segundo plano — sem nenhum elemento visual desenhado na tela para os usuários. Isso permitiu testar a infraestrutura sob carga real, com milhões de usuários reais, sem que ninguém percebesse, antes de finalmente "ligar" a interface visual do Chat em abril de 2008.

## 9.4 Google e Kubernetes: o Rolling Update como padrão de mercado

O Kubernetes, criado e aberto como projeto open-source pelo Google em 2014, tem suas raízes em um sistema interno do Google chamado Borg, usado havia mais de uma década para orquestrar as cargas de trabalho de serviços como Google Search, Gmail e YouTube. Ao projetar o Kubernetes, o Google incorporou diretamente a experiência acumulada com o Borg — e uma dessas decisões de design foi tornar o Rolling Update o comportamento padrão do objeto Deployment, exatamente pela combinação de segurança e simplicidade que essa estratégia oferece para a grande maioria das aplicações. É por isso que, hoje, praticamente qualquer empresa que rode Kubernetes está usando Rolling Update como sua estratégia de implantação primária, mesmo sem ter tomado uma decisão explícita nesse sentido — é simplesmente o que acontece "de fábrica".

## 9.5 O que acontece sem essas estratégias: o caso Knight Capital

### Estudo de caso de alerta

Este caso não trata de uma empresa de tecnologia usando essas estratégias com sucesso — pelo contrário, é um exemplo real e bem documentado do que pode acontecer quando um deploy é feito sem os cuidados discutidos neste guia.

Em 1º de agosto de 2012, a Knight Capital Group — então uma das maiores empresas de negociação automatizada de ações dos Estados Unidos, responsável por cerca de 10% de todo o volume negociado em ações listadas no país — perdeu aproximadamente 440 a 460 milhões de dólares em apenas 45 minutos, um valor superior ao lucro anual da empresa. A causa raiz, segundo o relatório oficial da SEC (a comissão de valores mobiliários dos EUA), foi um erro no processo de implantação de um novo sistema de roteamento de ordens.

Entre 27 e 31 de julho de 2012, a equipe da Knight implantou manualmente um novo código em oito servidores, um por dia, sem revisão por um segundo técnico e sem um processo automatizado de verificação. Um dos técnicos esqueceu de copiar o novo código para um dos oito servidores. Nesse servidor específico, permaneceu ativo um código antigo e obsoleto, chamado internamente de "Power Peg", que não era usado desde 2003, mas nunca havia sido removido — apenas desativado. O problema é que o novo código reaproveitava uma sinalização (flag) no sistema que, sem querer, reativava esse código morto.

Quando o mercado abriu, os sete servidores com o código correto funcionaram normalmente. O oitavo servidor, porém, começou a executar o algoritmo obsoleto do Power Peg, que comprava e vendia ações continuamente sem parar. Em 45 minutos, esse único servidor gerou mais de 4 milhões de execuções em 154

ações diferentes, acumulando posições indesejadas de bilhões de dólares. A empresa precisou de um resgate financeiro emergencial para não quebrar, e foi adquirida por uma concorrente poucos meses depois.



#### **A relação com o que você aprendeu neste guia**

O incidente reúne, em um único caso, quase todos os problemas que as estratégias deste guia foram criadas para evitar: um deploy manual e não padronizado (o oposto de um Rolling Update automatizado e verificado por health checks), a ausência de qualquer validação gradual com tráfego real antes do rollout completo (o oposto de um Canary), a impossibilidade de reverter instantaneamente porque o problema só apareceu em produção plena (o oposto do rollback imediato do Blue-Green), e código obsoleto deixado no sistema por quase uma década em vez de removido (exatamente o risco de dívida técnica descrito na Seção 6.3 sobre Feature Flags).

## **9.6 Fontes e leitura complementar**

Os relatos acima foram baseados nas seguintes fontes primárias, que podem ser consultadas para aprofundamento:

- Netflix Technology Blog — "Automated Canary Analysis at Netflix with Kayenta" ([netflixtechblog.com](https://netflixtechblog.com)).
- Netflix Technology Blog — "Full Cycle Developers at Netflix — Operate What You Build" ([netflixtechblog.com](https://netflixtechblog.com)).
- AWS Documentation — "Amazon ECS blue/green service deployments workflow" ([docs.aws.amazon.com](https://docs.aws.amazon.com)).
- AWS Whitepaper — "Blue/Green Deployments on AWS" ([docs.aws.amazon.com](https://docs.aws.amazon.com)).
- Meta/Facebook Engineering — "Facebook Chat" ([engineering.fb.com](https://engineering.fb.com), 2008).
- [featureflags.io](https://featureflags.io) — "Getting Started", com depoimento de Chad Dickerson (Etsy).
- U.S. Securities and Exchange Commission — Release No. 70694 (Ordem sobre o caso Knight Capital, 16 de outubro de 2013), conforme resumido por J. R. Varma e por Doug Seven em "Knightmare: A DevOps Cautionary Tale".

## 10. Glossário de termos

| Termo                                | Definição                                                                                                                                                  |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Artefato                             | O resultado empacotado de um processo de build — por exemplo, uma imagem Docker ou um arquivo executável — pronto para ser implantado.                     |
| CI/CD                                | Sigla geral para o conjunto de práticas de Integração Contínua e Entrega/Implantação Contínua.                                                             |
| Container (contêiner)                | Uma unidade padronizada de software que empacota o código e todas as suas dependências, para que a aplicação rode de forma confiável em qualquer ambiente. |
| Downtime                             | Período em que um sistema está indisponível ou não funcional para os usuários.                                                                             |
| Health check                         | Verificação automatizada e periódica de que uma instância de software está funcionando corretamente.                                                       |
| Instância / Réplica / Pod            | Uma cópia individual em execução de uma aplicação. "Pod" é o termo específico usado no Kubernetes.                                                         |
| Load balancer (balanceador de carga) | Componente que distribui as requisições recebidas entre múltiplas instâncias de uma aplicação.                                                             |
| Observabilidade                      | A capacidade de entender o estado interno de um sistema a partir de métricas, logs e traces expostos externamente.                                         |
| Orquestrador                         | Sistema responsável por gerenciar automaticamente o ciclo de vida de instâncias de aplicações (criar, monitorar, substituir, escalar) — ex.: Kubernetes.   |
| Pipeline                             | Uma sequência automatizada de etapas (build, teste, empacotamento, deploy) pela qual o código passa até chegar em produção.                                |
| Rollback                             | Ato de reverter um sistema para uma versão anterior, considerada estável.                                                                                  |
| Schema (de banco de dados)           | A estrutura formal de um banco de dados: tabelas, colunas, tipos de dados e relacionamentos.                                                               |
| SLA                                  | Service Level Agreement — acordo formal sobre o nível mínimo de disponibilidade ou desempenho esperado de um serviço.                                      |